

POO Challenge

O Cliente Não Sabe Programar

Desafio: vocês são uma equipe de desenvolvimento contratada para criar uma solução para um problema real. O cliente sabe o que precisa, mas não sabe como programar. Cabe à equipe analisar o problema, decidir como representá-lo e implementar uma solução funcional em C#.

1. Organização da competição

Equipes

6 equipes com aproximadamente 5 integrantes.

Tempo

10 minutos de planejamento e 50 minutos de desenvolvimento.

Apresentação

Cada equipe terá até 3 minutos para demonstrar sua solução.

Tecnologia

C# utilizando os conceitos estudados até o momento.

2. O problema

Sistema de Estacionamento

Uma empresa precisa de um pequeno sistema para controlar um estacionamento.

O sistema deve permitir controlar os veículos que entram e saem do estacionamento, registrar seus dados e calcular o valor que deverá ser pago.

A equipe deve analisar o problema e decidir quais classes, propriedades e métodos são necessários para representar essa situação.

3. Requisitos obrigatórios

O sistema desenvolvido pela equipe deverá atender aos seguintes requisitos:

1. Representação de um veículo

O sistema deverá possuir uma classe capaz de representar um veículo.

2. Dados do veículo

O veículo deverá possuir, no mínimo, informações para representar: **placa, modelo, horário de entrada, horário de saída e valor pago.**

3. Entrada

O sistema deverá permitir registrar a entrada de um veículo.

4. Saída

O sistema deverá permitir registrar a saída de um veículo.

5. Cálculo

O sistema deverá calcular o valor a ser pago pelo período de permanência.

6. Exibição

O sistema deverá conseguir exibir os dados de um veículo estacionado ou que tenha saído.

7. Encapsulamento

As informações do objeto não devem ser alteradas livremente pelo código externo. A equipe deverá decidir quais dados precisam de acesso controlado.

8. Validação

O sistema deverá impedir situações inválidas definidas nas regras deste desafio.

4. Regras do estacionamento

- 1 A placa não pode ser vazia.
- 2 Um veículo não pode possuir duas entradas simultaneamente.
- 3 A hora de saída não pode ser anterior à hora de entrada.
- 4 O valor cobrado é de **R\$ 10,00 por hora.**
- 5 Uma permanência inferior a uma hora deve ser cobrada como uma hora inteira.
- 6 O valor pago não pode ser alterado diretamente pelo programa principal.
- 7 Após a saída, o sistema deve conseguir informar a placa, o modelo, o tempo de permanência e o valor pago.

Exemplo

Entrada: 10:00

Saída: 10:30

Tempo considerado: 1 hora

Valor: R\$ 10,00

Atenção: os requisitos indicam o que o sistema precisa fazer, mas não dizem como vocês devem programá-lo. A equipe deverá tomar as decisões de modelagem.

5. Extra

Controle de vagas

Depois de implementar os requisitos anteriores, a equipe deverá adicionar uma nova funcionalidade:

O estacionamento possui capacidade para **5 veículos**.

1. O sistema deve conseguir armazenar os veículos atualmente estacionados.
2. Quando não houver vagas disponíveis, uma nova entrada deve ser recusada.
3. Quando um veículo sair, sua vaga deverá ficar disponível novamente.
4. O sistema deverá conseguir consultar os veículos atualmente estacionados.

Importante: esta etapa exige que vocês pensem em como trabalhar com vários objetos ao mesmo tempo. A forma de implementar isso fica a critério da equipe.

6. Antes de programar: planejem

Durante os primeiros 10 minutos, a equipe deverá analisar o problema e planejar a solução. Não comece simplesmente criando código.

Definam:

- Quais classes serão necessárias?
- Quais propriedades cada classe deverá possuir?
- Quais propriedades podem ser alteradas externamente?
- Quais propriedades precisam de acesso controlado?
- Quais informações podem ser calculadas?
- Quais métodos representam as ações do sistema?
- Quais situações devem ser consideradas inválidas?
- Como os vários veículos serão armazenados?

PLANEJAMENTO DA EQUIPE

Classes:

Principais propriedades:

Principais métodos:

Regras e validações:

7. O que será observado

A solução será analisada considerando principalmente:

- se o sistema funciona;
- se o problema foi bem compreendido;
- se as classes representam adequadamente o problema;
- se propriedades e métodos foram utilizados de maneira coerente;
- se o encapsulamento foi utilizado corretamente;
- se as regras e situações inválidas foram tratadas;
- se a solução consegue trabalhar com vários veículos;
- se a equipe consegue explicar as decisões tomadas.

8. Apresentação

A apresentação deverá ser feita diretamente a partir do programa desenvolvido. Não é necessário preparar slides.

Durante a apresentação, a equipe deverá demonstrar:

1. A criação de um veículo.
2. O registro de uma entrada.
3. O registro de uma saída.
4. O cálculo do valor.

5. Pelo menos uma situação inválida sendo impedida pelo sistema.
6. O controle das vagas.

9. Regra principal

Não existe uma única forma correta de modelar o problema. O objetivo é construir uma solução funcional e justificar as decisões tomadas pela equipe.

A equipe deve utilizar os conceitos estudados em aula e não simplesmente copiar uma solução pronta. **Usar IA ou programas prontos não será permitido e a equipe caso seja pega por outra, será desclassificada.**